

HBOS · 裸机内核操作系统

# HBOS 应用开发手册

HAX Application Development Guide · [.hax / C 版](#)

本手册随 HBOS 内核源码发布，供第三方应用开发者参考。

## HBOS APPLICATION DEVELOPER GUIDE

2026/06/30 · VERSION 1.2 · HBOS v0.1-beta3 · SDK: hax.h rev.3 (目录遍历 + 全屏画布 + 并发窗口)

# 目录 CONTENTS

## 第 1 章 — 概述、格式与类型系统

### 1.1 关于本手册

### 1.2 什么是 HAX 应用 (.hax)

#### 1.2.1 ELF64 结构

#### 1.2.2 .haxmeta 元数据段

### 1.3 自动导入机制

#### 1.3.1 构建流水线详解

#### 1.3.2 预编译应用的导入

### 1.4 编译 HAX 应用

#### 1.4.1 Makefile 自动构建

#### 1.4.2 手动编译步骤

#### 1.4.3 链接脚本与地址空间

### 1.5 专有类型系统

### 1.6 应用元数据宏 HAX\_APP()

## 第 2 章 — HAX SDK 参考手册

### 2.1 文本输入输出

### 2.2 文件操作

### 2.3 系统服务

### 2.4 错误处理约定

### 2.5 直接使用 POSIX libc

### 2.6 目录遍历 (opendir / readdir)

### 2.7 GUI 全屏画布接口

### 2.8 并发窗口接口 (推荐)

## 第 3 章 — 运行、发现与调试

### 3.1 TUI 与 GUI 类型说明

### 3.2 apps / run 命令

### 3.3 在图形桌面运行

### 3.4 参数传递

### 3.5 退出码约定

## 第 4 章 — 完整示例

### 4.1 hello —— 最小应用 (TUI)

### 4.2 guess —— 猜数字游戏 (GUI)

### 4.3 catf —— 读取并显示文件

### 4.4 ls —— 列出目录内容

### 4.5 paint —— GUI 全屏画布涂鸦

### 4.6 wdemo —— 并发窗口计数器

## 附录 A — 底层 POSIX 系统调用速查

## 附录 B — hax\_meta\_t 二进制布局

## 附录 C — 常见构建错误排查

# 第 1 章 OVERVIEW, FORMAT & TYPES

本章介绍 HAX 应用格式的设计目标、内部结构、自动导入机制、编译工具链以及类型系统。

## 1.1 关于本手册

本手册面向希望在 **HBOS**（裸机内核操作系统）上开发应用的用户。HBOS 提供一套名为 **HAX** (*HBOS Application eXecutable*) 的应用机制，设计目标是：

- **零注册**——把源码 (.c) 或预编译应用 (.hax) 放进 ./app 目录，执行一次 make，应用在系统启动后自动出现，无需修改内核代码或配置文件。
- **标准入口**——应用写 main(argc, argv)，与普通 C 程序没有区别。
- **轻量 SDK**——一个头文件 <hax.h> 覆盖 90% 的常用场景；更底层的需求可直接调用 POSIX libc。
- **可移植**——.hax 是标准 ELF64，可用任何支持 x86-64 裸机 freestanding 的工具链构建。

本手册中所有命令示例默认你已在 HBOS 系统提示符 (HBOS>) 或 HBOS 构建机 (宿主 Linux/x86-64)

下操作。

## 1.2 什么是 HAX 应用 (.hax)

### 1.2.1 ELF64 结构

一个 .hax 文件就是标准 **ELF64 可执行程序**——ELF Magic、Program Headers、.text/.data/.bss 段均与普通用户态 ELF 完全相同，可用 readelf -h foo.hax 验证。它以静态链接方式包含 HBOS 用户态 libc 与 crt0，运行时加载到地址 0x1000000000 (256 GiB)，通过 int 0x80 陷入内核进行系统调用。

### 1.2.2 .haxmeta 元数据段

与普通 ELF 的唯一区别是额外携带一个名为 .haxmeta 的只读段，其中存放一个 hax\_meta\_t 结构（见 1.6）。该段仅供 tools/genhax.py 在构建时读取，内核在运行时通过 blob 清单而非 ELF 段头来定位

元数据，因此此段不占用运行时内存。

扩展名 `.hax` 是构建系统的识别标志，与其他规范中的 `.epf/.elf` 等扩展名无关。

## 1.3 自动导入机制

### 1.3.1 构建流水线详解

核心逻辑：只要 `./app` 目录里存在 `.hax` 文件（编译产物或预编译），它就会被自动加入系统。

| 步骤   | 触发条件                             | 具体行为                                                                                                                                                  |
|------|----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------|
| ① 编译 | <code>app/*.c</code> 有更新         | Makefile 用 HAX 工具链把每个 <code>app/foo.c</code> 编译为 <code>build/app/foo.hax</code> （ELF64，含 <code>.haxmeta</code> ）                                      |
| ② 解析 | <code>build/app/*.hax</code> 有更新 | <code>tools/genhax.py</code> 依次解析每个 ELF 的 <code>.haxmeta</code> 段（magic 校验 → 提取 <code>kind/name/desc</code> ），若无合法元数据则以文件名兜底（ <code>kind=TUI</code> ） |
| ③ 打包 | 同上                               | 生成器按 16 字节对齐拼接所有 ELF 为 <code>build/hax_blob.bin</code> ，并输出 <code>build/hax_manifest.c</code> （含 <code>hax_app_table[]</code> + 计数）                   |
| ④ 嵌入 | <code>blob</code> 有更新            | <code>src/user/hax_blob.asm</code> 用 <code>incbin</code> 把 <code>blob</code> 嵌入内核只读数据段； <code>manifest</code> 编译进内核                                   |
| ⑤ 注册 | 系统启动                             | 内核通过 <code>hax_app_table[]</code> 查表； <code>apps</code> 命令列出， <code>run &lt;名&gt;</code> 从 <code>blob</code> 切片后 ELF 加载并生成任务                          |

步骤 ②③ 由同一次 Python 脚本完成（GNU Make &: grouped target），`blob` 与 `manifest` 原子更新，

不会出现两者不一致的情况。需 GNU Make ≥ 4.3（Ubuntu 22.04+ 默认满足）。

### 1.3.2 预编译应用的导入

你可以把别处交叉编译好的 `.hax`（只需是合法 ELF64 + `.haxmeta`，构建主机不限）直接放进 `./app`。执行 `make` 时，生成器会与 `./app/*.c` 产物一起打包。预编译文件无需改动 Makefile，也无需源码。

## 1.4 编译 HAX 应用

### 1.4.1 Makefile 自动构建

```
make          # 构建整个系统，含所有 .hax 自动打包

make hax-apps # 仅构建 ./app 下的应用，不重新链接内核
```

把源文件加入 `./app` 后不需要改任何 Makefile——通配符规则会自动发现新文件。

### 1.4.2 手动编译步骤

Makefile 内部等价命令如下，可用于调试或在其他构建系统中集成：

```
# 1. 编译目标文件

gcc -c -m64 -ffreestanding -fno-stack-protector -fno-pie \

    -mno-red-zone -mno-sse -mno-sse2 \

    -Iapp/include -Isrc/user -Isrc/user/libc \

    app/myapp.c -o build/app/myapp.o

# 2. 链接为 .hax（ELF64 静态）

ld -m elf_x86_64 -static -nostdlib -T src/user/user.ld \

    build/user/libc/*.o build/user/crt0.o \

    build/app/myapp.o -o build/app/myapp.hax

# 3. 打包（可选，单独重新打包）

python3 tools/genhax.py \

    --blob build/hax_blob.bin \
```

```
--manifest build/hax_manifest.c \

build/app/myapp.hax
```

1.4.3 链接脚本与地址空间

所有 HAX 应用共享链接脚本 `src/user/user.ld`，加载基地址固定为 `0x1000000000`（256 GiB）。内核在执行 `elf64_load_and_spawn` 时把各段映射到用户页表的对应位置；每个应用运行在独立地址空间（独立任务），互不干扰。

| 地址范围            | 用途                             |
|-----------------|--------------------------------|
| 0x1000000000+   | 用户态代码与数据（.text / .data / .bss） |
| 栈顶（由内核分配）       | 用户栈（初始 128 KiB）                |
| 0x0000 – 0x0FFF | 保留（NULL 陷阱页，访问触发 #PF 而非挂起）     |

1.5 专有类型系统

`<hax.h>` 定义了一组与平台无关的定宽类型，在 32/64 位工具链上都有相同语义：

| 类型   | 位宽 | 有符号号 | C 底层类型 (x86-64) | 典型用途          |
|------|----|------|-----------------|---------------|
| HI8  | 8  | 是    | signed char     | 字节序列、字符       |
| HU8  | 8  | 否    | unsigned char   | 原始字节、颜色分量     |
| HI16 | 16 | 是    | short           | 小范围整数         |
| HU16 | 16 | 否    | unsigned short  | 端口号、UTF-16 单元 |
| HI32 | 32 | 是    | int             | 通用整数、返回值      |
| HU32 | 32 | 否    | unsigned int    | 颜色值 0xRRGGBB  |
| HI64 | 64 | 是    | long long       | 文件偏移、大整数      |

|            |    |   |                    |                     |
|------------|----|---|--------------------|---------------------|
| HU64       | 64 | 否 | unsigned long long | 位掩码、地址              |
| HCOLOR     | 32 | 否 | unsigned int       | RGB 颜色 (0x00RRGGBB) |
| hax_meta_t | —  | — | struct 104 字节      | 应用元数据 (构建时用)        |

## 1.6 应用元数据宏 HAX\_APP()

在源文件**全局作用域**（任意位置，不得在函数内）调用一次 `HAX_APP()` 宏，声明该应用的名称、描述和类型：

```
HAX_APP("myapp", "我的第一个 HBOS 应用", HAX_KIND_TUI);
```

宏展开后会把一个 `hax_meta_t` 常量放进 `.haxmeta` ELF 段，标记为 `__attribute__((used))` 防止链接器优化掉。

| 参数   | 类型     | 约束                | 说明                                            |
|------|--------|-------------------|-----------------------------------------------|
| name | 字符串字面量 | ≤ 31 字节，<br>UTF-8 | 应用唯一标识符， <code>run</code> 命令用此名查找；建议全小写英文，无空格 |
| desc | 字符串字面量 | ≤ 63 字节，<br>UTF-8 | 一句话描述，显示在 <code>apps</code> 列表中               |
| kind | 常量     | 见下表               | 应用类型，影响启动器中的分类标签                              |

| kind 常量       | 值 | 含义               |
|---------------|---|------------------|
| HAX_KIND_TUI  | 1 | 终端文本界面应用（命令行运行）  |
| HAX_KIND_GUI  | 2 | 图形桌面应用（出现在桌面启动器） |
| HAX_KIND_BOTH | 3 | 两种入口均注册          |

每个 `.hax` 文件只能有一个 `HAX_APP()`。若检测到多个 `.haxmeta` 符号，构建将报错。



# 第 2 章 HAX SDK REFERENCE

SDK 是对 HBOS 用户态 libc 与 int 0x80 系统调用的轻量包装，让常见操作一行搞定。所有声明在 `<hax.h>`，无需额外链接标志。

## 2.1 文本输入输出

---

**`void hax_print(const char *s);`**

把字符串 `s` 写到标准输出，**不**追加换行。`s` 为 `NULL` 时行为未定义。

**`void hax_println(const char *s);`**

输出 `s` 后追加一个换行符 (`\n`)。等价于 `hax_print(s); hax_print("\n");`

**`void hax_printf(const char *fmt, ...);`**

格式化输出，语义同 C99 `printf`。支持格式说明符：

| 说明符                    | 类型                                | 输出                           |
|------------------------|-----------------------------------|------------------------------|
| <code>%d / %i</code>   | <code>int</code>                  | 有符号十进制                       |
| <code>%u</code>        | <code>unsigned int</code>         | 无符号十进制                       |
| <code>%x / %X</code>   | <code>unsigned int</code>         | 十六进制（小写/大写）                  |
| <code>%ld / %lu</code> | <code>long / unsigned long</code> | 64 位十进制                      |
| <code>%s</code>        | <code>char *</code>               | 字符串                          |
| <code>%c</code>        | <code>int</code>                  | 单字符                          |
| <code>%p</code>        | <code>void *</code>               | 指针（十六进制， <code>0x</code> 前缀） |
| <code>%%</code>        | —                                 | 字面量 <code>%</code>           |

```
hax_printf("进程 PID=%d, 名称=%s\n", hax_pid(), "myapp");
```

```
hax_printf("地址 0x%p, 计数=%lu\n", ptr, count);
```

### **int hax\_input(char \*buf, int cap);**

从标准输入读取一行到 buf（含 NUL 终止符，**不含**结尾换行）。

- 最多写入 cap - 1 个有效字符，第 cap 字节写 '\0'。
- 返回实际字符数（不含 NUL）；遇到 EOF 或错误返回 -1。
- buf 为 NULL 或 cap <= 0 时直接返回 -1。

```
char line[128];

hax_print("请输入指令: ");

int n = hax_input(line, sizeof(line));

if (n < 0) { hax_println("读取失败"); hax_exit(1); }

hax_printf("你输入了 %d 个字符: %s\n", n, line);
```

### **int hax\_getch(void);**

读取并返回一个字节（0-255）；无数据时返回 -1。该函数不回显字符、不等待换行，适合逐键处理。

```
hax_println("按任意键继续...");

while (hax_getch() < 0) hax_sleep(0); /* 轮询 */
```

## 2.2 文件操作

---

### **long hax\_read\_file(const char \*path, void \*buf, long cap);**

打开 path，读取全部内容到 buf（最多 cap 字节），关闭文件。

- 返回实际读到的字节数（文件长度 ≤ cap 时等于文件大小）。
- 失败（文件不存在、权限不足、路径为 NULL）返回 -1。

- 不追加 NUL，若需当字符串使用，请自行在返回值处置 '\0'。

```
char data[4096];

long n = hax_read_file("/etc/hostname", data, sizeof(data) - 1);

if (n >= 0) { data[n] = '\0'; hax_printf("主机名: %s\n", data); }
```

## long hax\_write\_file(const char \*path, const void \*buf, long len);

以覆盖创建模式打开 path，把 buf 的 len 字节写入，关闭文件。

- 返回实际写入字节数；失败返回 -1。
- 若文件不存在则创建；若已存在则截断后写入。

```
const char *msg = "Hello, HBOS!\n";

long w = hax_write_file("/tmp/test.txt", msg, 13);

hax_printf("写入 %ld 字节\n", w);
```

## 2.3 系统服务

| 函数签名                          | 返回值   | 说明                              |
|-------------------------------|-------|---------------------------------|
| void hax_sleep(unsigned sec); | 无     | 休眠整数秒。sec=0 为 yield（让出 CPU，不挂起） |
| void hax_exit(int code);      | 不返回   | 以状态码终止当前应用。code=0 表示成功，非零表示错误   |
| int hax_pid(void);            | 进程 ID | 返回当前任务的内核 PID（≥1 的正整数）          |

## 2.4 错误处理约定

SDK 函数遵循以下约定，与 POSIX libc 一致：

- 返回 int 或 long 的函数，失败时返回 -1（不设 errno，HBOS 当前不暴露全局 errno）。

- 返回指针的函数，失败时返回 **NULL**。
- `hax_exit` / `hax_sleep` 不会失败。

HBOS v0.1 暂不设 `errno`。若需区分具体错误原因，请使用底层 `open/read/write` 系统调用（见附录

A），它们通过返回值编码错误码（-ENOENT、-EACCES 等）。

## 2.5 直接使用 POSIX libc

SDK 是轻量封装，满足不了的场景可直接 `#include libc` 头文件：

```
#include <hax.h>

#include <libc/string.h> /* memcpy, strlen, strcmp ... */

#include <libc/stdlib.h> /* atoi, malloc, free ... */

#include <libc/stdio.h> /* printf, fopen, fread ... */

#include <libc/socket.h> /* socket, connect, send ... */
```

完整列表见附录 A。

## 2.6 目录遍历（`opendir` / `readdir`）

HBOS libc 提供 POSIX 风格的目录遍历接口，声明在 `<libc/dirent.h>`。

| 函数                                                            | 说明                                |
|---------------------------------------------------------------|-----------------------------------|
| <code>DIR *opendir(const char *path);</code>                  | 打开目录，返回目录流；失败返回 <b>NULL</b>       |
| <code>struct dirent *readdir(DIR *d);</code>                  | 读取下一项，返回内部缓冲指针；无更多项返回 <b>NULL</b> |
| <code>int closedir(DIR *d);</code>                            | 关闭目录流，释放资源                        |
| <code>int getdents(int fd, struct dirent *, unsigned);</code> | 底层接口：从目录 <code>fd</code> 一次性读取目录项 |

struct dirent 关键字段：d\_name（文件名，NUL 结尾）、d\_type（DT\_DIR 目录 / DT\_REG 普通文件）、d\_ino（inode 号）。

```
#include <libc/dirent.h>

DIR *d = opendir("/");

struct dirent *e;

while ((e = readdir(d)) != 0) {

    hax_printf("%s%s\n", e->d_name, e->d_type == DT_DIR ? "/" : "");

}

closedir(d);
```

实现细节：内核 getdents 单次调用即返回整个目录（不维护游标），opendir 因此一次性把目录项读入

内部 16 KiB 缓冲，readdir 在其上迭代。完整示例见 4.4 节。

## 2.7 GUI 全屏画布接口

声明 HAX\_KIND\_GUI（或 BOTH）的应用可从图形桌面启动并**直接绘制到帧缓冲**，而非仅在终端输出文本。本接口为**全屏即时模式**画布：每帧「清屏 → 绘制 → 提交 → 取输入」，应用运行期间独占整屏（桌面让位），退出后桌面恢复。适合全屏小游戏。若想要可与桌面/其他应用**同时显示、可拖动的窗口**，请用 2.8 节的并发窗口接口。

| 函数                                              | 说明                                       |
|-------------------------------------------------|------------------------------------------|
| <code>int hax_gui_begin(int *w, int *h);</code> | 探测 GUI 是否可用；可用返回 1 并写入画布宽高，否则返回 0（应回退文本） |
| <code>void hax_gui_clear(HCOLOR c);</code>      | 用颜色（0xRRGGBB）填满整个画布                      |

|                                                                              |                                 |
|------------------------------------------------------------------------------|---------------------------------|
| <code>void hax_gui_rect(int x,int y,int w,int h,HCOLOR c);</code>            | 填充矩形                            |
| <code>void hax_gui_text(int x,int y,const char*s,HCOLOR c,int scale);</code> | 绘制文本（UTF-8，scale $\geq 1$ 整数放大） |
| <code>void hax_gui_present(void);</code>                                     | 把画布提交到屏幕（绘制后必须调用才可见）            |
| <code>int hax_gui_pollkey(void);</code>                                      | 轮询一个按键，返回键值；无按键返回 -1            |
| <code>int hax_gui_pollmouse(int *x,int *y);</code>                           | 轮询鼠标，写入绝对坐标，返回按键位掩码（bit0=左键）    |

另有一组**扩展绘图原语**（纯 SDK 实现，基于 `hax_gui_rect`，无额外系统调用）：

| 函数                                                                       | 说明                 |
|--------------------------------------------------------------------------|--------------------|
| <code>void hax_gui_pixel(int x,int y,HCOLOR c);</code>                   | 画一个像素              |
| <code>void hax_gui_frame(int x,int y,int w,int h,int t,HCOLOR c);</code> | 矩形描边（线宽 t）         |
| <code>void hax_gui_line(int x0,int y0,int x1,int y1,HCOLOR c);</code>    | 直线（Bresenham）      |
| <code>void hax_gui_fill_circle(int cx,int cy,int r,HCOLOR c);</code>     | 实心圆（水平 span 填充，高效） |
| <code>void hax_gui_circle(int cx,int cy,int r,HCOLOR c);</code>          | 圆环描边（中点画圆法）        |

```
int w, h;

if (!hax_gui_begin(&w, &h)) { hax_println("需要从桌面启动"); return 1; }

for (;;) {

    hax_gui_clear(0x202830);

    hax_gui_rect(20, 20, 120, 48, 0x14A6E0);

    hax_gui_text(28, 32, "Hello", 0xFFFFFFFF, 2);

    hax_gui_present();
}
```

```

int k = hax_gui_pollkey();

if (k == 'q' || k == 27) break;  /* q 或 ESC 退出 */

hax_sleep(0);          /* 让出 CPU */

}

```

全屏画布运行期间，桌面合成器自动让位给应用（应用通过 `hax_gui_begin` 登记为屏幕所有者）；应用

退出后桌面立即恢复。若需与桌面共存的窗口，见 2.8。

## 2.8 并发窗口接口（推荐）

并发窗口接口让应用拥有一个**独立窗口**，与桌面、其他应用窗口**同时显示**，可拖动、可关闭。每个窗口拥有自己的离屏表面，由桌面合成器每帧贴合到屏幕；应用与桌面在 100 Hz 抢占式调度下**并发运行**（应用在后台持续刷新时，桌面时钟、其他窗口照常工作）。这是编写图形应用的推荐方式。

| 函数                                                                 | 说明                                                |
|--------------------------------------------------------------------|---------------------------------------------------|
| <code>int hax_win_open(const char*title,int w,int h);</code>       | 打开窗口（内容区 $w \times h$ ），返回窗口 id ( $\geq 0$ ) 或 -1 |
| <code>int hax_win_active(int*w,int*h);</code>                      | 窗口是否仍活动：是返回 1 并写入当前 $w$ 、 $h$ ；被关闭返回 0            |
| <code>void hax_win_clear(HCOLOR c);</code>                         | 用颜色填满窗口                                           |
| <code>void hax_win_fill(int x,int y,int w,int h,HCOLOR c);</code>  | 窗口内填充矩形（坐标相对内容区）                                  |
| <code>void hax_win_text(int x,int y,const char*s,HCOLOR c);</code> | 窗口内绘制文本                                           |
| <code>void hax_win_present(void);</code>                           | 提交一帧（让出 CPU，使合成器尽快显示）                             |
| <code>int hax_win_poll(int*ev4);</code>                            | 取事件到 <code>ev[4]={type,a,b,c}</code> ，返回类型        |

(HAX\_EV\_\*, 0=无)

`void hax_win_close(void);`

关闭并销毁窗口

事件类型：`HAX_EV_KEY` (ev[1]=键值)、`HAX_EV_MOUSE` (ev[1]=x ev[2]=y ev[3]=按键位)、`HAX_EV_CLOSE` (用户点了关闭按钮，应退出)。

```
int w, h;

if (hax_win_open("我的应用", 360, 240) < 0) return 1;

while (hax_win_active(&w, &h)) {    /* 窗口被关闭时返回 0 */

    hax_win_clear(0x202830);

    hax_win_text(20, 20, "Hello", 0xFFFFFFFF);

    hax_win_present();

    int ev[4];

    int t = hax_win_poll(ev);

    if (t == HAX_EV_KEY && ev[1] == 'q') break;

    if (t == HAX_EV_CLOSE) break;

    hax_sleep(0);

}

hax_win_close();
```

**实现：**合成器在桌面合成阶段把每个窗口表面贴到屏幕并加标题栏/边框；输入由桌面路由到聚焦窗口的

事件队列。应用退出后窗口自动回收。最多 4 个并发窗口，单窗口最大 900×640。窗口应用应从 **开始菜**

**单**或在 GUI 终端用 run 启动（均为非阻塞）。



# 第 3 章 RUN, DISCOVERY & DEBUG

## 3.1 TUI 与 GUI 类型说明

| kind          | 标签      | 启动方式                    | 输入输出                |
|---------------|---------|-------------------------|---------------------|
| HAX_KIND_TUI  | TUI     | run <名>, 或桌面终端窗口        | stdin/stdout 重定向到终端 |
| HAX_KIND_GUI  | GUI     | 桌面启动器图标, 或 run <名><br>> | 当前版本: 输出到终端窗口       |
| HAX_KIND_BOTH | TUI GUI | 两种入口均可用                 | 同上                  |

GUI 应用绘制有三种方式: (1) 只输出文本 (在终端窗口显示); (2) 全屏画布 hax\_gui\_\* (见 2.7, 独占

整屏, 适合全屏游戏); (3) 并发窗口 hax\_win\_\* (见 2.8, 独立窗口, 与桌面/其他应用同时显示、可拖

动, 推荐)。后两者均直接绘制到帧缓冲。

## 3.2 apps / run 命令

```
HBOS> apps

hello - 最小 HAX 示例: 问候并回显参数 [TUI.hax]

guess - 猜数字小游戏 (1-100) [GUI.hax]

catf - 读取并显示文件内容 [TUI.hax]

ls - 列出目录下的文件与子目录 [TUI.hax]

paint - GUI 画布示例: 鼠标涂鸦 [GUI.hax]

wdemo - 并发窗口示例: 计数器 [GUI.hax]
```

```
HBOS> run hello
```

你好, HBOS! 这是一个 .hax 应用。

我的 PID 是 7

```
HBOS> run hello 一 二 三
```

你好, HBOS!

收到参数: 一 二 三

#### 命令

#### 作用

`apps`

列出所有已注册应用（内建命令 + .hax 应用），显示名称、描述、类型标签

`run <名> [参数...]`

运行应用；优先匹配内建命令，未命中后查找 .hax 表，再未命中报错

### 3.3 在图形桌面运行

在 HBOS 图形桌面中，打开"终端"窗口后即可像命令行一样输入 `apps` 与 `run` 命令。此外，`HAX_KIND_GUI`（或 `BOTH`）类型的应用会**自动追加到开始菜单**（已固定应用网格的内置项之后），点击图标即可启动，等价于 `run <名>`——这正是 GUI 画布应用（如 `paint`）的推荐启动方式。

### 3.4 参数传递

`run myapp a b c` 等价于 C 程序的 `main(4, {"myapp", "a", "b", "c", NULL})`：

- `argv[0]` 始终为应用名（与 `HAX_APP` 中的 `name` 相同）。
- `argv[1..argc-1]` 为用户传入的额外参数，均为 NUL 结尾 UTF-8 字符串。
- `argv[argc]` 保证为 `NULL`（标准 POSIX 约定）。
- 参数最多 31 个（含 `argv[0]`），超出部分截断。

### 3.5 退出码约定

---

| 退出码  | 含义                            |
|------|-------------------------------|
| 0    | 成功                            |
| 1    | 一般错误（参数错误、IO 失败等）             |
| 2    | 使用方法错误（参数数量不对）                |
| 127  | 命令未找到（由 shell/run 命令返回，非应用本身） |
| 其他非零 | 应用自定义错误码                      |

内核通过 `task_wait` 获取退出码并传回 `run` 命令；当前 shell 不打印退出码，可用 `hax_printf` 在退出前自行输出。

# 第 4 章 COMPLETE EXAMPLES

## 4.1 hello —— 最小应用 (TUI)

---

演示：HAX\_APP()、文本输出、参数处理、hax\_pid()。

```
/* app/hello.c */

#include <hax.h>

HAX_APP("hello", "最小 HAX 示例：问候并回显参数", HAX_KIND_TUI);

int main(int argc, char **argv) {

    hax_println("你好，HBOS! 这是一个 .hax 应用。");

    hax_printf("我的 PID 是 %d\n", hax_pid());

    if (argc > 1) {

        hax_print("收到参数:");

        for (int i = 1; i < argc; i++)

            hax_printf(" %s", argv[i]);

        hax_println("");

    }

    return 0;

}
```

把文件放入 `./app/hello.c`，执行 `make`，重启后：

```
HBOS> run hello 世界
```

```
你好，HBOS! 这是一个 .hax 应用。
```

```
我的 PID 是 5
```

```
收到参数: 世界
```

## 4.2 guess —— 猜数字游戏（GUI）

---

演示：`HAX_KIND_GUI`、循环输入、`atoi`、`hax_exit`。

```
/* app/guess.c */

#include <hax.h>

#include <libc/stdlib.h> /* atoi */


HAX_APP("guess", "猜数字小游戏（1-100）", HAX_KIND_GUI);


int main(int argc, char **argv) {

    (void)argc; (void)argv;


    /* 以 PID 为简易随机种子，避免每次都一样 */

    int secret = (hax_pid() * 37 + 11) % 100 + 1;

    char line[32];


    hax_println("我想了一个 1-100 的整数，猜猜看！（输入 q 退出）");
```

```
for (int tries = 1; ; tries++) {

    hax_printf("第 %d 次猜测> ", tries);

    if (hax_input(line, sizeof(line)) < 0 || line[0] == 'q')

        break;


    int v = atoi(line);

    if (v < 1 || v > 100) {

        hax_printf("请输入 1-100 之间的整数。");

        tries--;

        continue;

    }


    if (v < secret) hax_printf("太小了，再大一点。");

    else if (v > secret) hax_printf("太大了，再小一点。");

    else {

        hax_printf("猜对了！ 答案是 %d，你用了 %d 次。 \n",

            secret, tries);

        return 0;

    }

}
```

```
hax_printf("游戏结束。正确答案是 %d。 \n", secret);

return 0;

}
```

## 4.3 catf —— 读取并显示文件（TUI）

演示：参数处理、`hax_read_file`、标准 libc 调用（`atoi`）、错误处理与退出码。

```
/* app/catf.c */

#include <hax.h>

HAX_APP("catf", "读取并显示文件内容", HAX_KIND_TUI);

int main(int argc, char **argv) {

    if (argc < 2) {

        hax_println("用法: run catf <文件路径>");

        return 2;          /* 用法错误 */

    }

    static char buf[8192];

    long n = hax_read_file(argv[1], buf, sizeof(buf) - 1);

    if (n < 0) {

        hax_printf("无法读取文件: %s\n", argv[1]);
```

```

    return 1;          /* 一般错误 */

}

buf[n] = '\0';        /* 当字符串处理 */

hax_print(buf);

if (n > 0 && buf[n - 1] != '\n')

    hax_println("");    /* 补一个换行 */

hax_printf("\n[共 %ld 字节]\n", n);

return 0;

}

HBOS> run catf /etc/hostname

hbos

[共 5 字节]

```

本示例只用了 `hax_read_file`。目录遍历见下一节。

## 4.4 ls —— 列出目录内容（TUI）

演示：`opendir/readdir/closedir`、`d_type` 判断、参数处理。

```

/* app/ls.c */

#include <hax.h>

#include <libc/dirent.h>

```



```
HAX_APP("ls", "列出目录下的文件与子目录", HAX_KIND_TUI);
```

```
int main(int argc, char **argv) {
```

```
    const char *path = (argc > 1) ? argv[1] : "/";
```

```
    DIR *d = opendir(path);
```

```
    if (!d) { hax_printf("无法打开目录: %s\n", path); return 1; }
```

```
    hax_printf("目录 %s:\n", path);
```

```
    struct dirent *ent;
```

```
    int files = 0, dirs = 0;
```

```
    while ((ent = readdir(d)) != 0) {
```

```
        if (ent->d_name[0] == '\0') continue;
```

```
        if (ent->d_type == DT_DIR) { hax_printf(" [DIR] %s\n", ent->d_name); dirs++; }
```

```
        else { hax_printf("    %s\n", ent->d_name); files++; }
```

```
    }
```

```
    closedir(d);
```

```
    hax_printf("共 %d 个文件， %d 个目录。 \n", files, dirs);
```

```
    return 0;
```

```
}
```

## 4.5 paint —— GUI 画布涂鸦（GUI）

演示：`hax_gui_begin` 探测、鼠标轮询绘制、按键退出、无 GUI 时回退。从图形桌面启动；按住左键涂鸦，`c` 清屏，`q`/`ESC` 退出。

```
/* app/paint.c */

#include <hax.h>

HAX_APP("paint", "GUI 画布示例：鼠标涂鸦", HAX_KIND_GUI);

int main(int argc, char **argv) {

    (void)argc; (void)argv;

    int w, h;

    if (!hax_gui_begin(&w, &h)) {

        hax_println("paint 需要从图形桌面启动。");

        return 1;

    }

    hax_gui_clear(0x10141A);

    hax_gui_rect(0, 0, w, 28, 0x14A6E0);

    hax_gui_text(8, 6, "HBOS Paint - 左键涂鸦 c 清屏 q 退出", 0xFFFFFFFF, 1);

    hax_gui_present();

    for (;;) {
```

```

int mx, my;

int btn = hax_gui_pollmouse(&mx, &my);

if ((btn & 1) && my > 28) {    /* 左键拖动绘制 */

    hax_gui_rect(mx - 4, my - 4, 8, 8, 0xF5C842);

    hax_gui_present();

}

int k = hax_gui_pollkey();

if (k == 'q' || k == 27) break;

hax_sleep(0);

}

return 0;

}

```

## 4.6 wdemo —— 并发窗口计数器（GUI 窗口）

演示：并发窗口接口、事件循环、与桌面同时运行。窗口里的计数器持续自增（体现并发刷新），左键点击窗口 +10，按 **q** 或点标题栏关闭按钮退出。

```

/* app/wdemo.c（节选） */

#include <hax.h>

HAX_APP("wdemo", "并发窗口示例：计数器", HAX_KIND_GUI);

int main(int argc, char **argv) {

```

```
(void)argc; (void)argv;

int w, h;

if (hax_win_open("wdemo", 320, 220) < 0) return 1;


int count = 0, tick = 0;

while (hax_win_active(&w, &h)) {

    if (++tick % 8 == 0) count++;    /* 持续刷新，体现并发 */


    int ev[4], t = hax_win_poll(ev);

    while (t != HAX_EV_NONE) {

        if (t == HAX_EV_CLOSE) goto done;

        if (t == HAX_EV_KEY && (ev[1] == 'q' || ev[1] == 27)) goto done;

        if (t == HAX_EV_MOUSE && (ev[3] & 1)) count += 10;

        t = hax_win_poll(ev);

    }


    hax_win_clear(0x121820);

    hax_win_fill(0, 0, w, 30, 0x14A6E0);

    hax_win_text(10, 8, "并发窗口计数器", 0xFFFFFFFF);

    /* ...把 count 格式化为字符串后绘制... */

    hax_win_present();
```

```
    hax_sleep(0);

}

done:

    hax_win_close();

    return 0;

}
```

提示：把这些示例放进 `./app`，执行 `make`，无需修改任何其他文件即可看到它们出现在 `apps` 列表（GUI 类还会出现在桌面启动器）中。

# 附录 A POSIX SYSCALL QUICK REFERENCE

除 SDK 外，HAX 应用可直接使用 HBOS 用户态 libc 暴露的 POSIX 接口（声明在 `src/user/libc/` 下各头文件）。HBOS 支持约 62 个系统调用，覆盖以下类别：

| 类别     | 接口（常用子集）                                                                                          |
|--------|---------------------------------------------------------------------------------------------------|
| 文件 I/O | <code>open close read write lseek fstat stat unlink rename</code>                                 |
| 进程控制   | <code>getpid getppid exit sleep usleep fork execve waitpid kill</code>                            |
| 文件系统   | <code>mkdir rmdir getcwd chdir access ftruncate readlink opendir readdir closedir getdents</code> |
| 内存管理   | <code>sbrk brk mmap munmap mprotect malloc free realloc calloc</code>                             |
| 网络     | <code>socket bind listen accept connect send recv setsockopt</code>                               |
| 标准 I/O | <code>printf fopen fclose fread fwrite fgets fputs fseek ftell</code>                             |
| 字符串    | <code>strlen strcpy strncpy strcmp strncmp strcat strchr strstr memcpy memset memmove</code>      |
| 转换     | <code>atoi atol strtol strtoul itoa sprintf snprintf</code>                                       |

网络 DNS：HBOS 在网络配置未提供 DNS 地址时自动回退公共解析器（8.8.8.8），因此即便“静态 IP 未

填 DNS”，connect 域名参数与高层 `dns_resolve` 接口仍可正常工作。

# 附录 Bhax\_meta\_t BINARY LAYOUT

下表为 `hax_meta_t` 在 `.haxmeta` 段中的精确二进制布局，供需要手工构造或解析元数据的工具开发者参考。

| 偏移（字节） | 大小 | 字段       | 说明                                                             |
|--------|----|----------|----------------------------------------------------------------|
| 0      | 4  | magic    | 固定值 <code>0x4D584148</code> （小端），ASCII <code>HAXM</code>       |
| 4      | 4  | kind     | <code>1</code> =TUI, <code>2</code> =GUI, <code>3</code> =BOTH |
| 8      | 32 | name[32] | NUL 结尾 UTF-8 应用名，不足 32 字节用 NUL 补齐                              |
| 40     | 64 | desc[64] | NUL 结尾 UTF-8 描述，不足 64 字节用 NUL 补齐                               |

总大小：**104 字节**，结构体无填充（`__attribute__((packed))` 默认无需指定，字段已自然对齐）。

验证命令（宿主 Linux）：

```
readelf -S build/app/hello.hax | grep haxmeta

# 应看到 .haxmeta PROGBITS ... 104 字节

python3 - <<'EOF'

import struct, sys

data = open("build/app/hello.hax","rb").read()

# 搜索 magic

idx = data.find(b'HAXM')

if idx < 0: print("无 .haxmeta"); sys.exit(1)
```

```
magic, kind = struct.unpack_from("<II", data, idx)

name = data[idx+8:idx+40].rstrip(b'\x00').decode()

desc = data[idx+40:idx+104].rstrip(b'\x00').decode()

print(f"kind={kind} name={name!r} desc={desc!r}")

EOF
```



# 附录 CTROUBLESHOOTING

以下为构建和运行 HAX 应用时的常见问题及解决方法。

| 症状                                                    | 可能原因                              | 解决方法                                                           |
|-------------------------------------------------------|-----------------------------------|----------------------------------------------------------------|
| make: *** No rule to make target 'build/hax_blob.bin' | ./app 目录为空，<br>genhax.py 没有输入文件   | 至少放一个 .c 或预编译 .hax 到 ./app                                     |
| genhax.py: magic mismatch                             | 手写的 .hax 缺少合法<br>.haxmeta 段       | 检查 HAX_APP() 宏是否在全局作用域；或接受 genhax.py 的文件名兜底行为                  |
| run myapp: not found                                  | 应用名与 HAX_APP 第一参数不一致，或 make 未重新打包 | 确认 name 参数；执行 make 后重建 ISO 并重启                                 |
| 应用运行后立即退出（无输出）                                        | ELF 加载失败（地址冲突、段大小溢出）或 main 未被链接进去 | 用 readelf -l build/app/xxx.hax 检查 PT_LOAD 段；确认 crt0 + main 均链接 |
| ld: cannot find -lhax                                 | 使用了 -lhax 链接标志<br>（HAX SDK 不是共享库） | 删除 -lhax；SDK 通过 #include <hax.h> 内联或编译进 libc 目标文件              |
| Make grouped target 报错                                | GNU Make 版本 < 4.3（&: 语法不支持）       | make --version 确认版本；<br>Ubuntu 22.04+ 自带 Make 4.3+             |